



# Apache Kafka — Day 1

Fundamentals, Architecture & Cluster Management

## Duration

8 Hours

## Format

Concepts + Architecture + Demonstrations + Hands-on Labs

# Day 1: Learning Objectives

## → Messaging Systems

Explain why messaging systems are needed; differentiate P2P and Pub/Sub

## → Kafka's Role

Explain Kafka's role in event streaming, Big Data and ETL

## → Core Architecture

Understand brokers, topics, partitions, producers, consumers, offsets and consumer groups

## → Cluster Management

Explain replication, Leader/Follower, ISR, ZooKeeper, and single/multi-broker environments

## Today's Journey

01

## Messaging

02

## Kafka

03

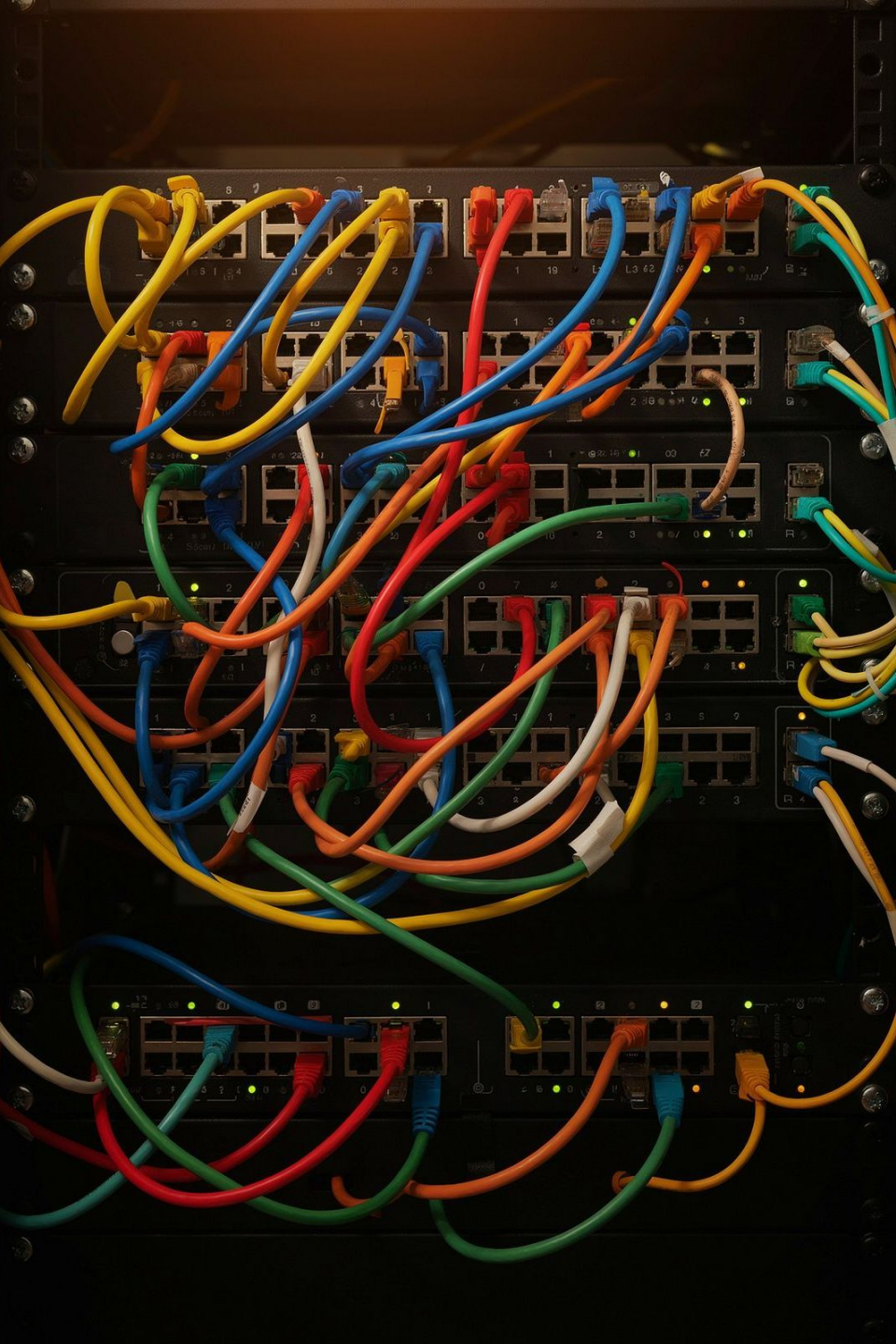
## Architecture

04

## Cluster

05

## Hands-on



# The Problem: Why Do We Need Messaging?

Consider an e-commerce **Order Service** that must directly call Payment, Inventory, Notification, Fraud, and Analytics. With direct synchronous integration, services become tightly coupled, failures propagate, every integration needs retry/error handling, scaling becomes difficult, and adding new consumers requires changing existing integrations.

## Tight Coupling

Services depend directly on each other's availability

## Failure Propagation

One failure can cascade across all downstream systems

## The Solution

**Producer → Messaging Platform → Consumer(s)** — publish without knowing every downstream implementation

# Messaging Patterns: P2P and Publish/Subscribe

## Point-to-Point

Producer → Queue → Consumer

Each message is processed by one logical consumer.

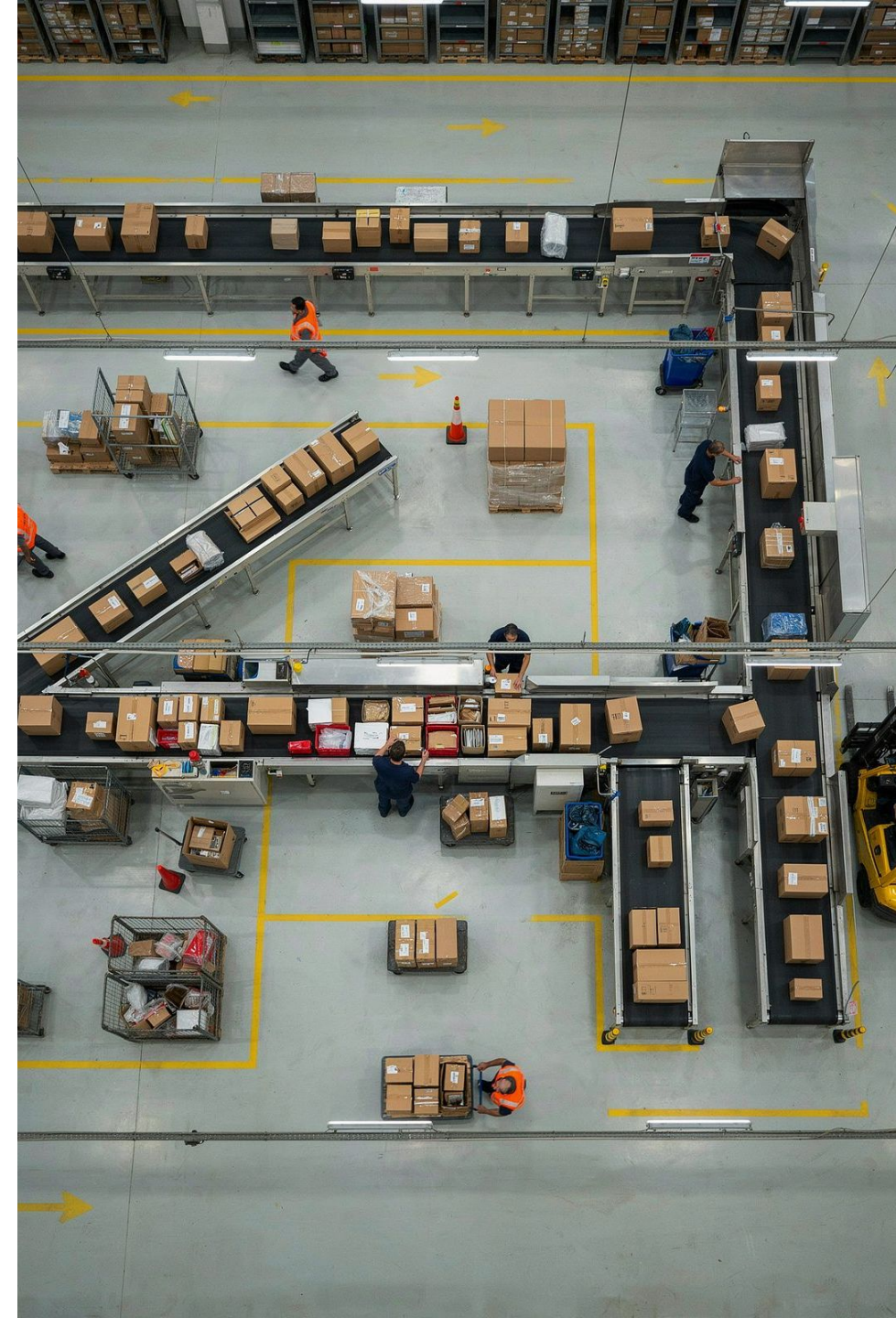
- Background jobs
- Invoice generation
- Email processing
- Task distribution

## Publish/Subscribe

Producer → Topic → Multiple Subscribers

Multiple subscribers independently react to the same event.

- An **OrderCreated** event may need to be processed simultaneously by Inventory, Analytics, and Notification services.



# Messaging Technologies: JMS, ActiveMQ, RabbitMQ & Kafka

## JMS

Java messaging API specification

## ActiveMQ

Traditional enterprise message broker

## RabbitMQ

Message broker with queue/routing patterns

## Kafka

Distributed event streaming platform

## Traditional Messaging

Send → Process → Acknowledge

## Kafka's Durable Event Log

Publish → Store → Consume → Replay

Replay capability is critical for analytics, integration, and data pipelines.

# Why Kafka?

Kafka addresses several enterprise requirements simultaneously — making it the platform of choice for modern event-driven systems.



## Scalability

Topics divided into partitions; workload distributed across brokers



## Performance

Designed for high-throughput event processing



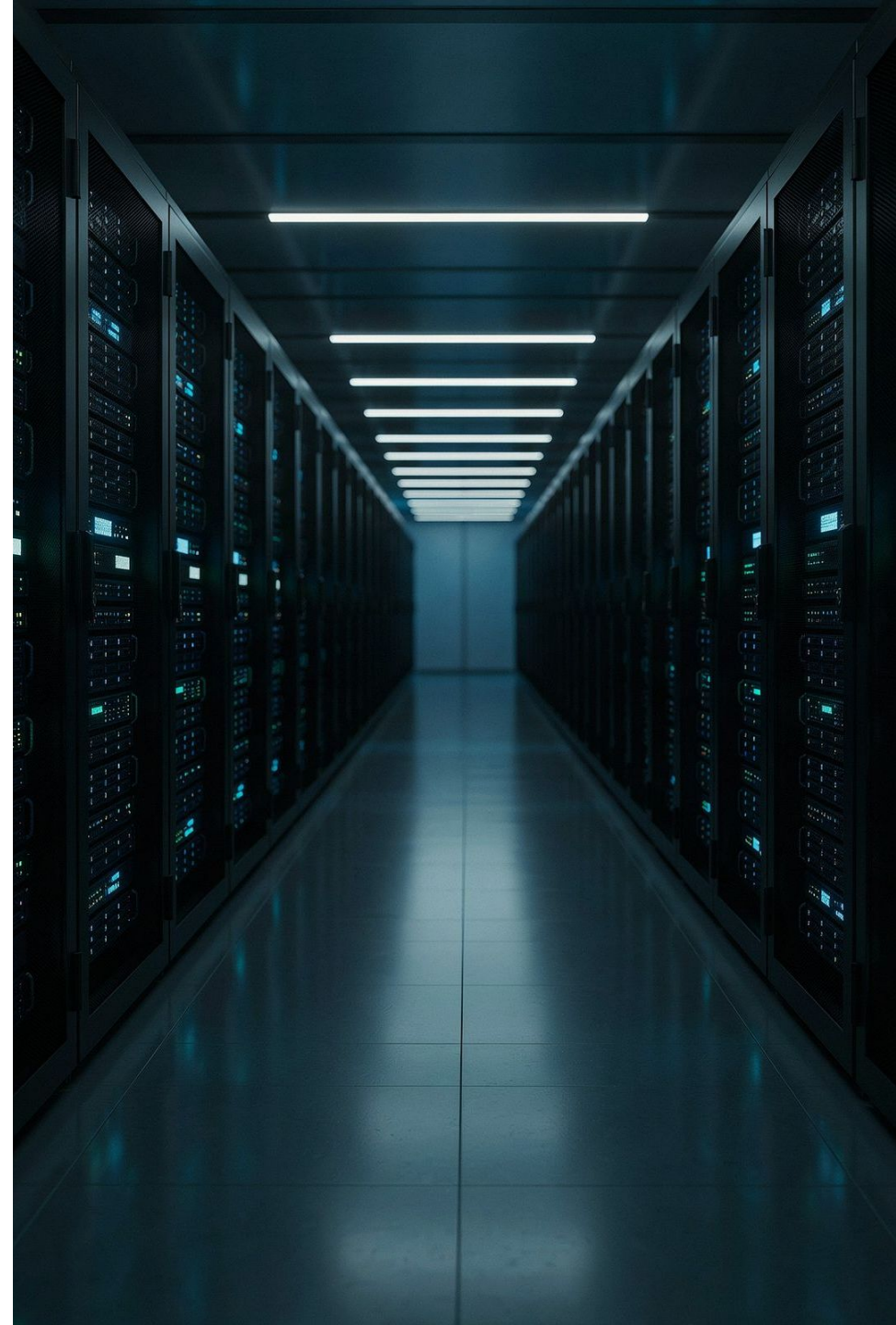
## Durability & Fault Tolerance

Events persisted; partitions replicated across brokers



## Decoupling & Replay

Producers don't need to know all consumers; retained events can be reread





# Kafka in Big Data, Data Ingestion & ETL

## Traditional ETL

Database → Extract → Transform → Load → Data Warehouse

Often **batch-oriented**, introducing latency between source and destination.

## Kafka-Based Ingestion

Applications, Databases, IoT Devices, and Logs all feed into **Kafka** as a continuous event layer, then fan out to:

- Data Lake
- Data Warehouse
- Stream Processing
- Analytics & ML

# Kafka Use Cases



## Event-Driven Architecture

OrderCreated → Payment → Shipping



## Financial Processing

Transaction → Fraud Detection → Analytics → Reporting



## Log Aggregation

Applications → Kafka → Monitoring/Search



## IoT

Devices → Kafka → Real-time Monitoring



## Clickstream

Website Events → Kafka → Customer Analytics



## Data Integration

Operational Systems → Kafka → Data Platform

# Kafka Core Components: First Look

## Core Flow

1

**Producers**

2

**Kafka Cluster**

Broker 1 · Broker 2 · Broker 3

3

**Consumer Groups**

## Supporting Concepts

### Topic

Named stream of events  
(e.g. orders)

### Partition

P0 · P1 · P2 within a topic

### Offset

Position of a record within  
a partition

### Replication

Leader · Follower · ISR

📌 This mental model — **Producer → Topic → Partition → Broker → Consumer** — is the foundation for the rest of Day 1.

# Hands-on Lab 1: Install & Run Kafka

**Objective:** Build your first working Kafka pipeline

## 1 Prepare & Start

Prepare the Kafka environment and start the broker

## 2 Create Topic

Verify broker availability and create a topic

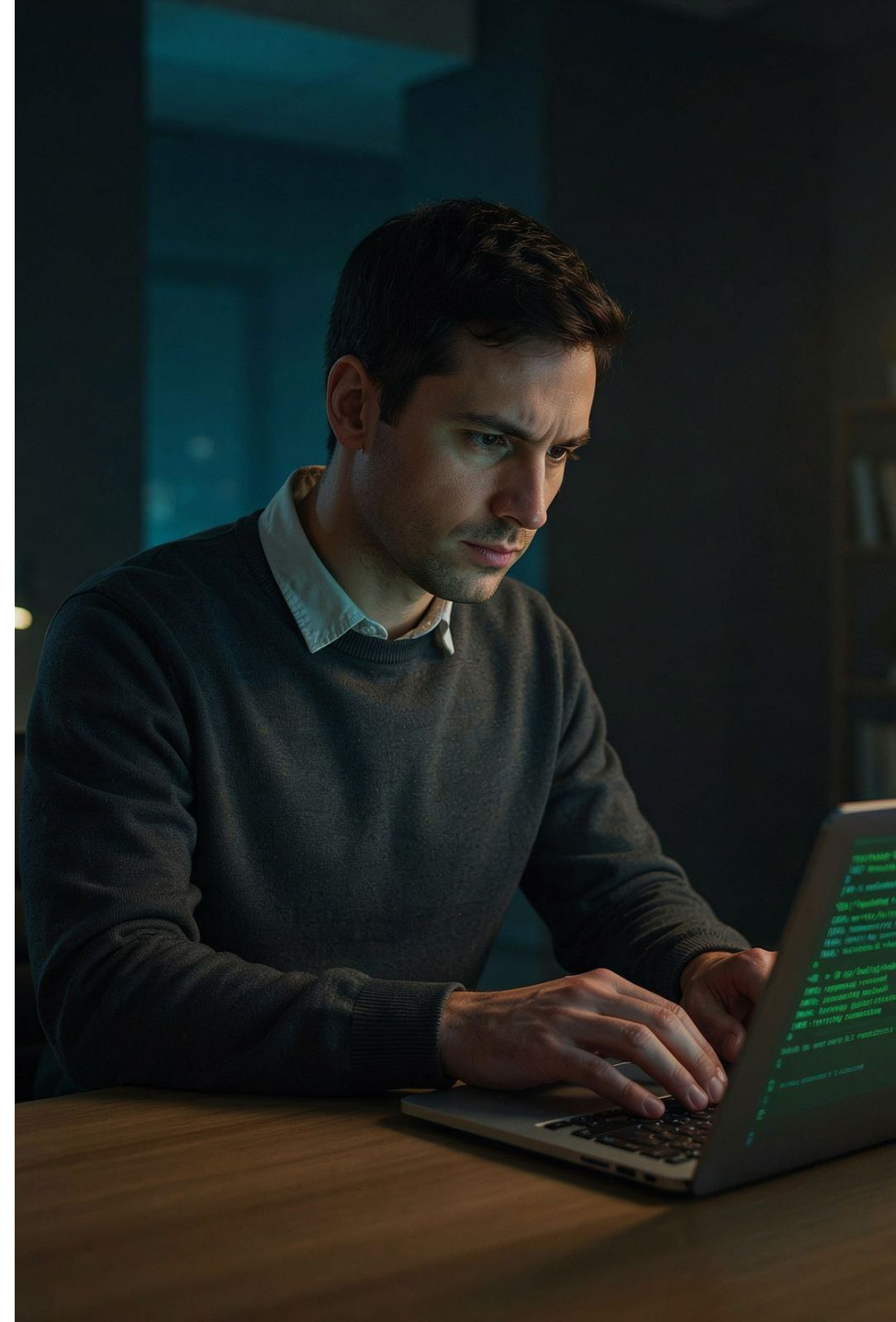
## 3 Produce Events

Start a console producer and publish sample events

## 4 Consume & Verify

Start a console consumer and verify messages are received

**Outcome:** Producer → Kafka Broker → Topic → Consumer — your first working pipeline.



# Kafka Cluster Architecture

## What is a Kafka Cluster?

A Kafka cluster contains one or more **brokers**. Each broker is a Kafka server responsible for:

- Receiving producer requests
- Storing partition data
- Serving consumers
- Participating in replication

Multiple brokers provide distributed storage and processing, enabling fault tolerance and horizontal scale.

Broker 1

Broker 2

Broker 3



# Topics & Partitions

## Topic

A named stream/category of events. Examples: orders, payments, customer-events, application-logs.

## Why Partitions?

- Horizontal scalability
- Parallel processing
- Data distribution across brokers

## Topic: orders — Partition Layout

### Partition 0

O1 → O4 → O7

### Partition 1

O2 → O5 → O8

### Partition 2

O3 → O6 → O9

❏ **Critical:** Ordering is maintained *within* a partition — this is why partition strategy matters.

# Producer & Message Flow

## A Producer publishes records to Kafka

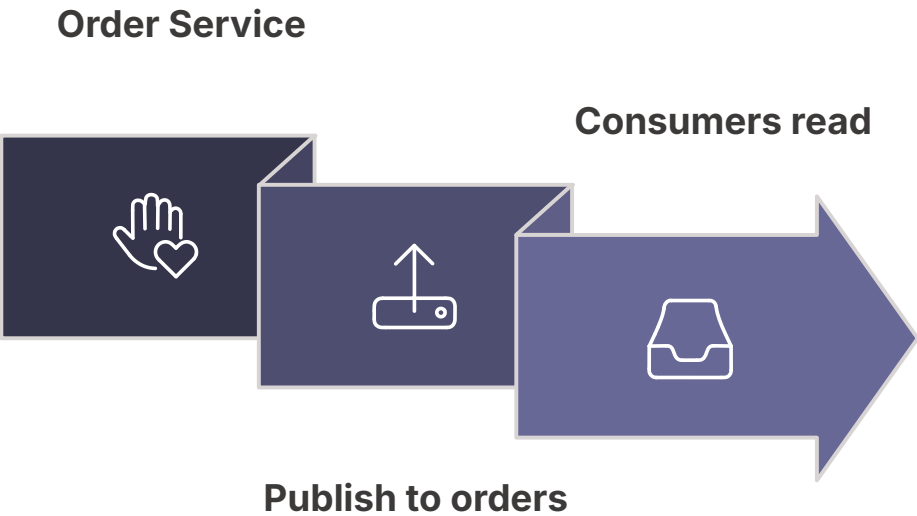
A Kafka record can contain:

|           |         |
|-----------|---------|
| Key       | Value   |
| Timestamp | Headers |

Example record:

```
Key = CUSTOMER-101
Value = {
  "orderId": 5001,
  "amount": 12500
}
```

## Flow



The **key** can influence which partition a record is routed to.



# Why Kafka Message Keys Matter

When `key = customerId`, events for the same customer consistently reach the **same partition**, preserving per-customer ordering.

## Consistent Routing

Customer-101 events → always Partition 1. Multiple events for the same key land together.

## Banking Example

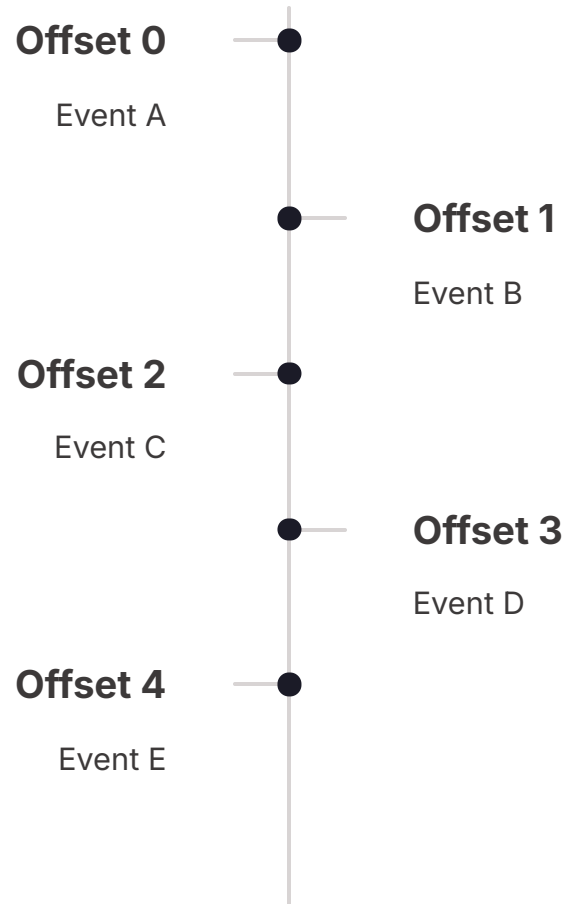
Account 101: Deposit → Withdraw → Transfer. If ordering matters for an entity, choosing an appropriate key is critical.

## Design Implication

Key selection is an architectural decision — choose a key that reflects the entity whose ordering must be preserved.

# Consumer, Offset & Replay

## Offsets in Partition 0



## What Offsets Enable



### Restart & Recovery

Resume from last committed offset



### Replay

Reprocess historical events



### Independent Progress

Each consumer tracks its own offset

 An offset is meaningful only within a specific partition.

# Consumer Groups & Scaling

## Work-Sharing Within a Group

Within one consumer group, each partition is assigned to **one consumer at a time**.

### Topic: orders

P0 —→ Consumer 1

P1 —→ Consumer 2

P2 —→ Consumer 3

### Group: order-service

## Independent Application Groups

Multiple groups each independently process the full topic — combining work-sharing *and* Pub/Sub behavior.

**Group: Inventory**

**Group: Analytics**

**Group: Notification**



# Replication: Leader, Follower & ISR

## Without Replication

Partition P0 lives only on Broker 1 — a single point of failure.

## With Replication

### Broker 1 — Leader ✓ ISR

Handles all read/write requests for the partition

### Broker 2 — Follower ✓ ISR

Replicates leader data

### Broker 3 — Follower ✓ ISR

Replicates leader data

## Key Concepts

### Leader

Handles all normal requests for the partition

### Followers

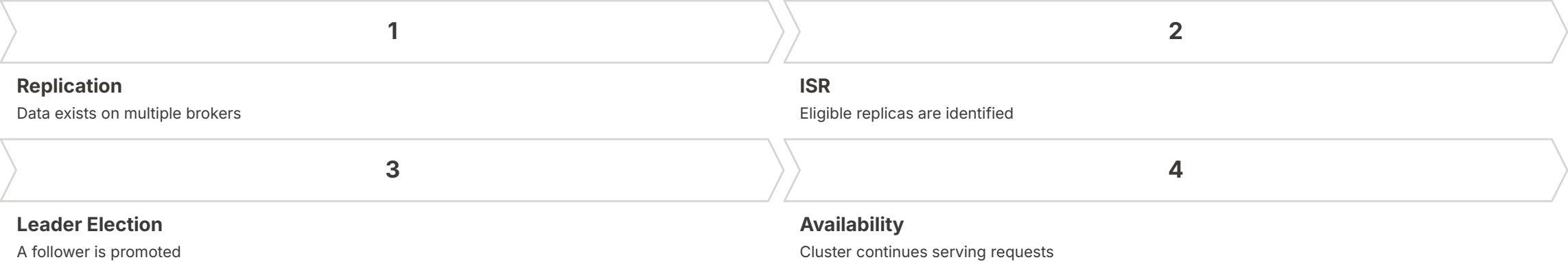
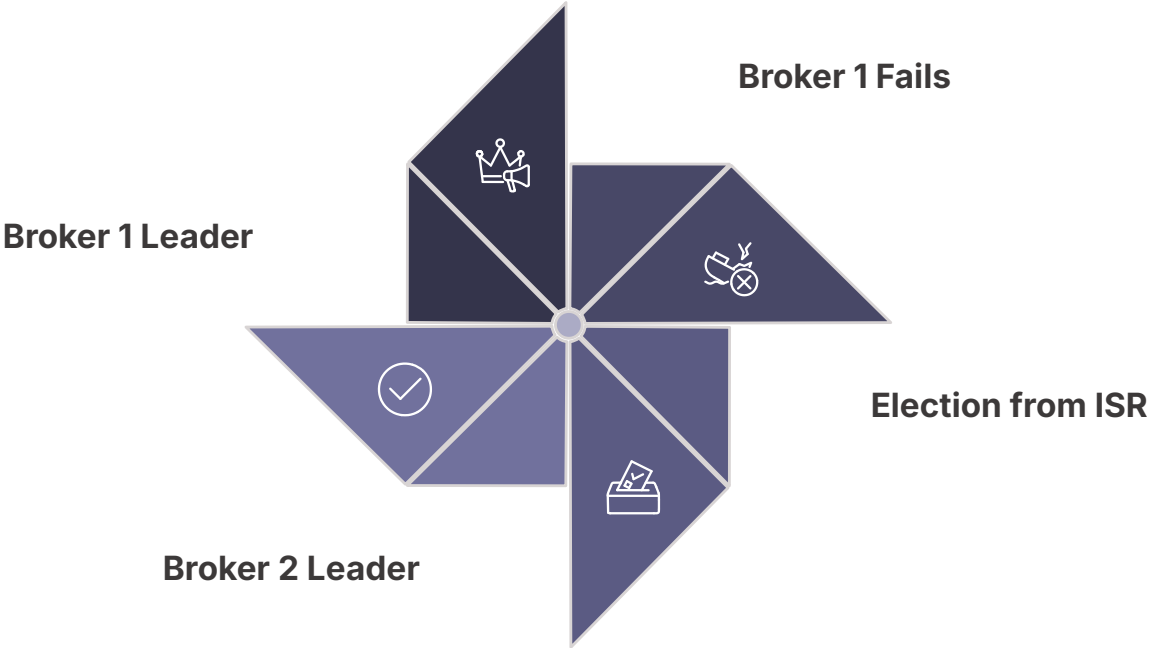
Replicate the leader's data passively

### ISR

In-Sync Replicas — sufficiently synchronized with the leader

Replication provides **fault-tolerance** capabilities for the cluster.

# What Happens When a Broker Fails?



# Complete Kafka Architecture

## Producer

Publishes records to topics

## Topic / Partition

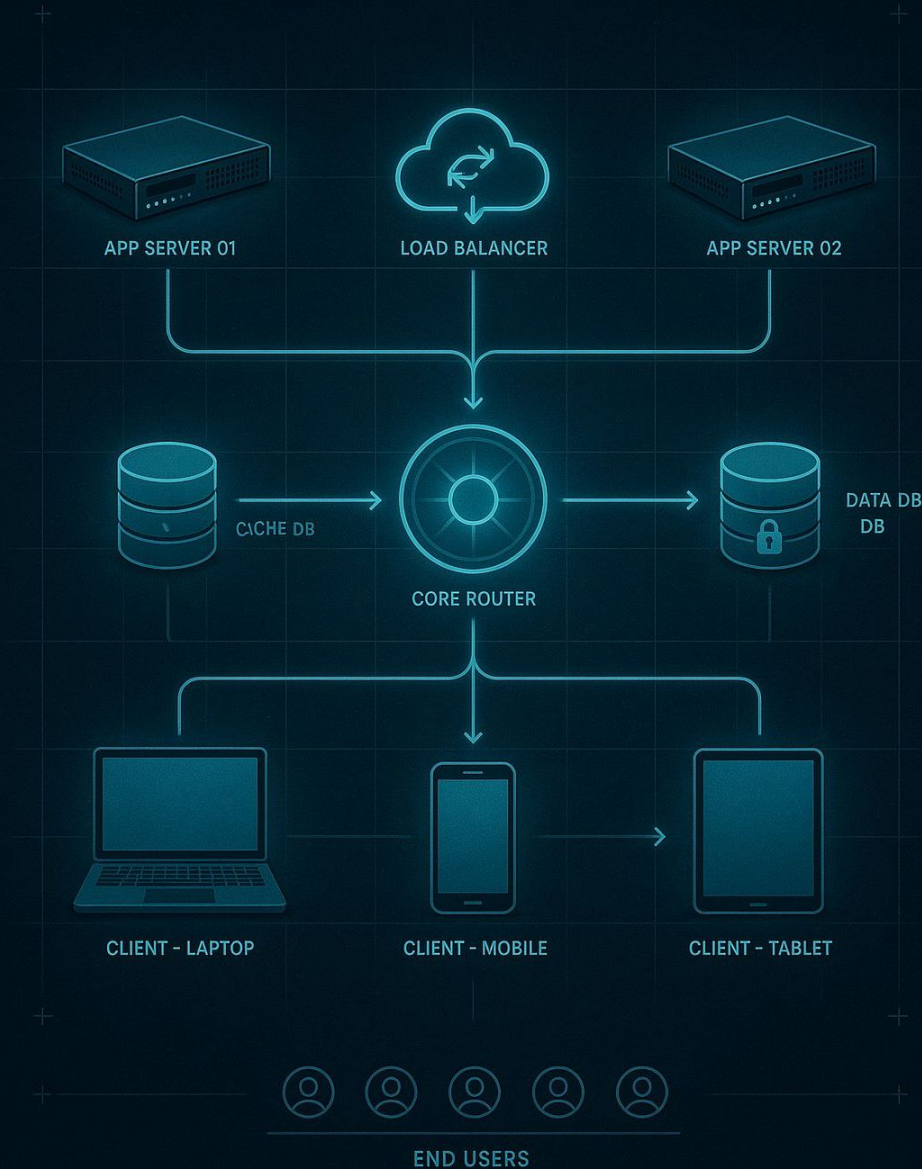
P0 Leader on Broker 1; P1 Leader on Broker 2; replicas distributed

## Consumer Group

C1, C2, C3 each assigned partitions; offsets tracked independently

📄 **End-to-end model:** Producer → Topic → Partition → Broker → Consumer Group → Offset, with Replication → Leader → Follower → ISR

## DISTRIBUTED SYSTEM ARCHITECTURE — SCHEMATIC v2.4



# Role of ZooKeeper

## Cluster Coordination

ZooKeeper acts as the coordination layer for Kafka cluster management. Kafka brokers register with and communicate through the ZooKeeper ensemble.

### Cluster Coordination

Manages broker membership and state

### ZooKeeper Ensemble

Multiple ZooKeeper nodes avoid a single point of failure

### Broker Relationship

Brokers register and report health to ZooKeeper

## Architecture

Kafka Cluster

Broker 1

Broker 2

Broker 3



# Hands-on Lab 2: Kafka CLI Exploration

Connect CLI output with the architecture learned in Slides 10–19

## CLI Tools

**kafka-topics**

**kafka-console-producer**

**kafka-console-consumer**

**kafka-consumer-groups**

## Exercise Steps

1. Create topic `orders` with 3 partitions
2. Publish Order-101 through Order-104
3. Inspect topic config, partitions, Leader, Replicas, ISR
4. Monitor consumer group progress

# Kafka Cluster Deployment Models



## Model 1 — Single Node / Single Broker

One VM, one broker. Ideal for learning and local development.



## Model 2 — Single Node / Multiple Brokers

One VM running Broker 1 (:9092), Broker 2 (:9093), Broker 3 (:9094). Explore cluster behavior locally without multiple machines.



## Model 3 — Multiple Nodes / Multiple Brokers

Node 1 → Broker 1, Node 2 → Broker 2, Node 3 → Broker 3. Demonstrates true distributed Kafka architecture (e.g. EC2 instances).

# Single Node, Multiple Broker Architecture

## One Machine — Three Brokers

ONE VM / MACHINE

Broker 1 :9092

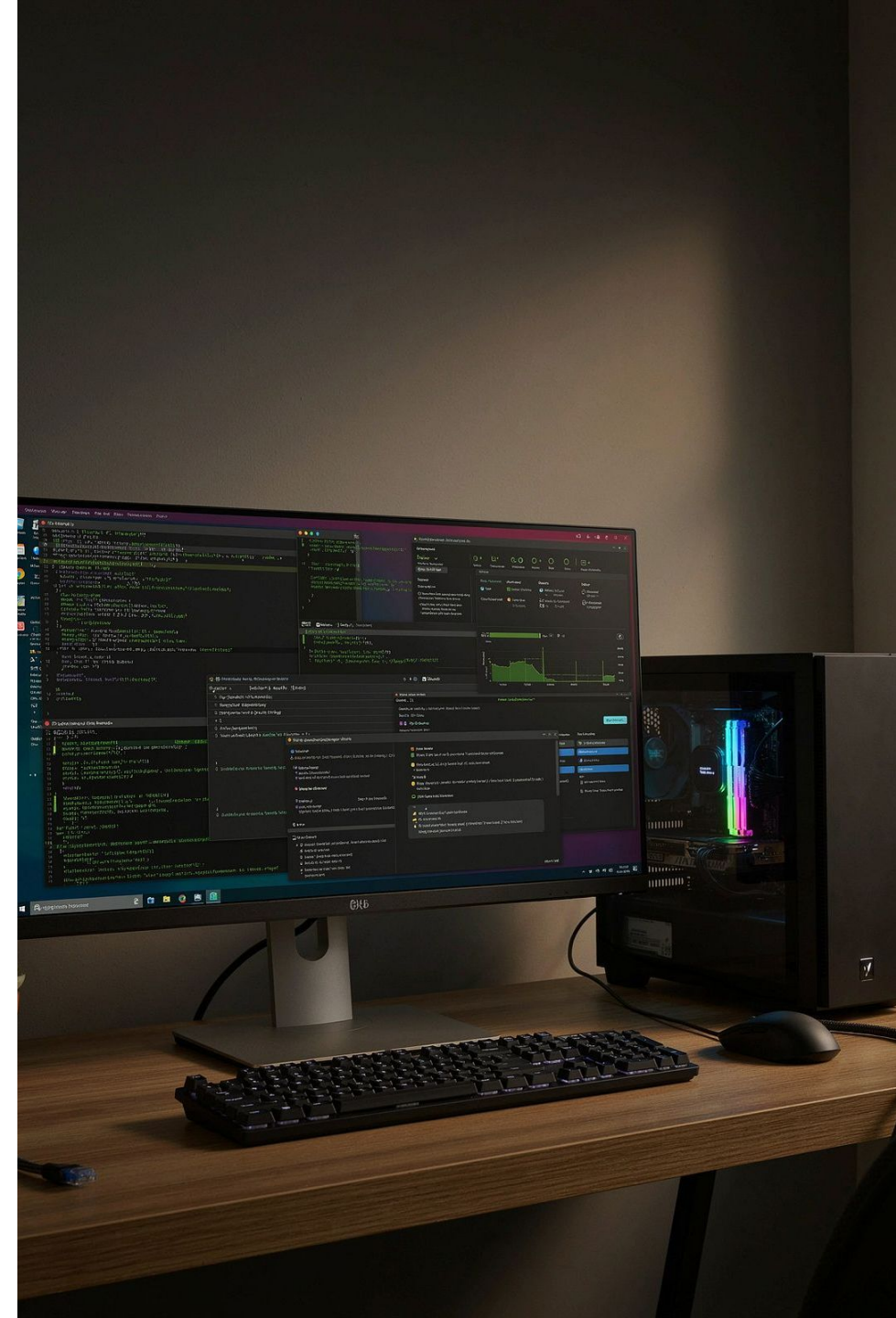
Broker 2 :9093

Broker 3 :9094

## What Students Can Explore

- Multiple broker configuration
- Partition placement across brokers
- Replication between brokers
- Leader/follower role assignment
- Simulated broker failure

All of this without requiring multiple physical servers — a practical learning environment.



# Multiple Nodes, Multiple Brokers

Node 1 — Broker 1

Node 2 — Broker 2

Node 3 — Broker 3

Topic partitions and replicas are now distributed across **separate machines**.

## Discussion Questions

- What happens if one broker stops?
- What happens if one machine fails?
- What happens if a partition leader disappears?

📄 Connect answers back to **Replication** and **ISR** — the concepts from the architecture section.



# ZooKeeper Cluster

## Why a ZooKeeper Cluster?

Distributed coordination itself should avoid a single point of failure. A ZooKeeper **ensemble** of multiple nodes provides high availability for cluster coordination.

## Architecture

Kafka Cluster

Broker 1 Broker 2 Broker 3

\ | /

ZooKeeper Cluster

Kafka brokers register with and rely on ZooKeeper for leader election, configuration, and membership tracking.

# Hands-on Lab 3: Single Node, Multiple Brokers

**Objective: Configure and run three brokers on one machine**

## 1 Configure Brokers

Set up unique broker IDs and ports for Broker 1, 2, and 3

## 2 Start & Verify

Start all brokers and verify the cluster is healthy

## 3 Create Topic

Create a multi-partition topic with replication configured

## 4 Observe & Produce

Describe topic — observe Leader/Replica/ISR — produce and consume messages



# Hands-on Lab 4: Broker Failure & Recovery

## Initial State

Topic: orders

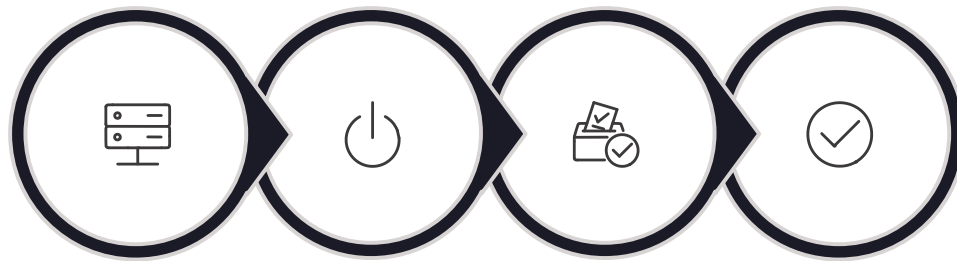
P0 → Leader: Broker 1

Replica: Broker 2

Replica: Broker 3

## Experiment

Stop Broker 1. Run the topic description again. Observe leader election.



Leader:  
Broker 1

Broker 1 Fails

Replica  
Election

New Leader  
from ISR

## Discussion Questions

- Which broker became the new leader?
- What happened to the ISR list?
- Can producers continue publishing?
- Can consumers continue reading?
- What happens when the old broker returns?

📌 This single exercise reinforces **Broker + Partition + Replication + Leader + Follower + ISR** together.

# Day 1 Enterprise Case Study

## Real-Time Banking Transaction Platform

Mobile, ATM, POS, and Web all publish to `transaction-events` → consumed independently by Fraud, Analytics, and Reporting consumer groups.

|   |                                                                                                                              |
|---|------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Topic Design</b><br>Use <code>transaction-events</code> as the topic name                                                 |
| 2 | <b>Partitions</b><br>Multiple partitions enable parallel processing and scalability                                          |
| 3 | <b>Message Key</b><br><code>accountId</code> — preserves per-account ordering where required                                 |
| 4 | <b>Consumer Groups</b><br>Fraud and Analytics must be <b>separate groups</b> — each independently requires every transaction |
| 5 | <b>Replication</b><br>Tolerate broker failures without data loss                                                             |

# Day 1 Recap & Knowledge Check

## Our Journey

### Messaging Systems

P2P & Pub/Sub · JMS /  
RabbitMQ / ActiveMQ

### Architecture

Broker · Topic · Partition ·  
Producer · Consumer ·  
Offset · Consumer Group

### Cluster Deployment

Single & Multi-Broker ·  
Broker Failure & Recovery

### Why Kafka?

Big Data · ETL · Features &  
Use Cases

### Replication

Leader · Follower · ISR ·  
ZooKeeper

## Final Knowledge Check

- Why Kafka instead of direct service-to-service integration?
- Topic vs. partition — what's the difference?
- Why do partitions enable scalability?
- What is an offset and why does it matter?
- How does a consumer group work?
- Why do we need replication?
- Leader vs. follower — what does ISR represent?
- What happens when a broker fails?
- Where does Kafka fit in an ETL/data pipeline?
- Single-node/multi-broker vs. multi-node/multi-broker?